

系统架构设计

历年真题考情：

本章节每年考察20分左右。

第二版更新：

第二版教材对应第7，8两章，是整个架构考试里最重要的章节，其内容不仅在选择题占大头，案例论文也是每年都考。本章节选择题，除了教材上有的架构基本概念、架构风格、DSSA、ABSD、质量属性和架构评估外，我们还将构件、中间件等相关题型也放到这里了，其对应的知识点应该是基于构件的软件工程那里的，大家注意综合学习。

软件架构概念

ABSD

软件架构风格

软件架构复用

DSSA

质量属性

架构评估

文老师软考教育

软件架构的概念

文老师软考教育

◆一个程序和计算系统软件体系结构是指系统的一个或者多个结构。结构中包括软件的构件，构件的外部可见属性以及它们之间的相互关系。

◆体系结构并非可运行软件。确切地说，它是一种表达，使软件工程师能够：

- (1) 分析设计在满足所规定的需求方面的有效性；
- (2) 在设计变更相对容易的阶段，考虑体系结构可能的选择方案；
- (3) 降低与软件构造相关联的风险。

◆上面的定义强调在任意体系结构的表述中“软件构件”的角色。软件构件简单到可以是程序模块或者面向对象的类，也可以扩充到包含数据库和能够完成客户与服务器网络配置的“中间件”。

◆软件体系结构的设计两个层次：数据设计和体系结构设计。数据设计体现传统系统中体系结构的数据构件和面向对象系统中类的定义（封装了属性和操作），体系结构设计则主要关注软件构件的结构、属性和交互作用。

◆建立体系结构层“内聚的、良好设计的表示”所需的方法，其目标是提供一种导出体系结构设计的系统化方法，而体系结构设计是构建软件的初始蓝图。

软件架构的概念

◆软件架构设计与生命周期

1. 需求分析阶段。需求分析和SA设计面临的是不同的对象：一个是问题空间；另一个是解空间。从软件需求模型向SA模型的转换主要关注两个问题：如何根据需求模型构建SA模型。如何保证模型转换的可追踪性。

◆针对这两个问题的解决方案，因所采用的需求模型的不同而异。在采用Use Case图描述需求的方法中，从Use Case图向SA模型（包括类图等）的转换一般经过词法分析和一些经验规则来完成，而可追踪性则可通过表格或者Use Case Map等来维护。

◆从软件复用的角度看，SA影响需求工程也有其自然性和必然性，已有系统的SA模型对新系统的需求工程能够起到很好的借鉴作用。在需求分析阶段研究SA，有助于将SA的概念贯穿于整个软件生命周期，从而保证了软件开发过程的概念完整性，也易于维护各阶段的可追踪性。

2. 设计阶段。是SA研究关注的最早和最多的阶段，这一阶段的SA研究主要包括：SA模型的描述、SA模型的设计与分析方法，以及对SA设计经验的总结与复用等。有关SA模型描述的研究分为3个层次：

（1）SA的基本概念：即SA模型由哪些元素组成，这些组成元素之间按照何种原则组织。包括构件和连接子。

（2）体系结构描述语言ADL：支持构件、连接子及其配置的描述语言就是如今所说的体系结构描述语言。ADL对连接子的重视成为区分ADL和其他建模语言的重要特征之一。典型的ADL包括UniCon、Rapide、Darwin、Wright、G2 SADL、Acme、xADL、XYZ/ADL和ABC/ADL等。

软件架构的概念

（3）SA模型的多视图表示：从不同的视角描述特定系统的体系结构，从而得到多个视图，并将这些视图组织起来以描述整体的SA模型。典型的包括4+1模型（逻辑视图、进程视图、开发视图、物理视图，加上统一的场景）、Hofmesiter的4视图模型（概念视图、模块视图、执行视图和代码视图）、CMU-SEI的Views and Beyond模型（模块视图、构件和连接子视图、分配视图）等。

3. 实现阶段。最初SA研究往往只关注较高层次的系统设计、描述和验证。为了有效实现SA设计向实现的转换，实现阶段的体系结构研究表现在以下几个方面。

（1）研究基于SA的开发过程支持，如项目组织结构、配置管理等。

（2）寻求从SA向实现过渡的途径，如将程序设计语言元素引入SA阶段、模型映射、构件组装、复用中间件平台等。

（3）研究基于SA的测试技术。

◆SA引入能够有效扩充现有配置管理的能力，通过在SA描述中引入版本、可选择项等信息，可以分析和记录不同版本构件和连接子之间的演化，从而可用来组织配置管理的相关活动。为了填补高层SA模型和底层实现之间的鸿沟，可通过封装底层的实现细节、模型转换、精化等手段缩小概念之间的差距。典型的方法如下。

（1）在SA模型中引入实现阶段的概念，如引入程序设计语言元素等。

（2）通过模型转换技术，将高层的SA模型逐步精化成能够支持实现的模型。

（3）封装底层的实现细节，使之成为较大粒度构件，在SA指导下通过构件组装的方式实现系统，这往往需要底层中间件平台的支持。

软件架构的概念

4. 构件组装阶段。在SA设计模型的指导下，可复用构件的组装可以在较高层次上实现系统，并能够提高系统实现的效率。在构件组装的过程中，SA设计模型起到了系统蓝图的作用。研究内容包括如下两个方面。

- (1) 如何支持可复用构件的互联，即对SA设计模型中规约的连接子的实现提供支持。
- (2) 在组装过程中，如何检测并消除体系结构失配问题。

◆中间件遵循特定的构件标准，为构件互联提供支持，并提供相应的公共服务，如安全服务、命名服务等。中间件支持的连接子实现的优势：中间件提供了构件之间跨平台交互的能力，且遵循特定的工业标准；产品化的中间件可以提供强大的公共服务能力，这样能够更好地保证最终系统的质量属性。

◆在构件组装阶段的失配问题主要包括：由构件引起的失配、由连接子引起的失配、由于系统成分对全局体系结构的假设存在冲突引起的失配等。

5. 部署阶段。SA对软件部署作用如下。

- (1) 提供高层的体系结构视图来描述部署阶段的软硬件模型。
- (2) 基于SA模型可以分析部署方案的质量属性，从而选择合理的部署方案。

◆现阶段，基于SA的软件部署研究更多地集中在组织和展示部署阶段的SA、评估分析部署方案等方面，部署方案的分析往往停留在定性的层面，并需要部署人员的参与。

软件架构的概念

6. 后开发阶段。是指软件部署安装之后的阶段。这一阶段的SA研究主要围绕维护、演化、复用等方面来进行。典型的研究方向包括动态软件体系结构、体系结构恢复与重建等。

- (1) 动态软件体系结构。现实中的软件具有动态性，体系结构会在运行时发生改变。

◆运行时变化包括两类：软件内部执行所导致的体系结构改变（很多服务器端软件会在客户请求到达时创建新的构件来响应用户的请求）；软件系统外部的请求对软件进行的重新配置（高安全性软件系统在升级或进行其他修改时不能停机）。

◆包括两个部分的研究：

◆体系结构设计阶段的支持：主要包括变化的描述、如何根据变化生成修改策略、描述修改过程、在高抽象层次保证修改的可行性以及分析、推理修改所带来的影响等。

◆运行时刻基础设施的支持：主要包括系统体系结构的维护、保证体系结构修改在约束范围内、提供系统的运行时刻信息、分析修改后的体系结构符合指定的属性、正确映射体系结构构造元素的变化到实现模块、保证系统的重要子系统的连续执行并保持状态、分析和测试运行系统等。

(2) 体系结构恢复与重建。对于现有系统在开发时候没有考虑SA的情况，从这些系统中恢复或重购体系结构。从已有的系统中获取体系结构的重建方法分为4类：手工体系结构重建、工具支持的手工重建、通过查询语言来自动建立聚集、使用其他技术（如数据挖掘等）。

软件架构的概念

软件架构的重要性

1. 架构设计能够满足系统的品质
2. 架构设计使受益人达成一致的目标
3. 架构设计能够支持计划编制过程
4. 架构设计对系统开发的指导性
5. 架构设计能够有效地管理复杂性
6. 架构设计为复用奠定了基础
7. 架构设计能够降低维护费用
8. 架构设计能够支持冲突分析

基于架构的软件开发

◆ABSD方法是由**体系结构驱动的**，即指由**构成体系结构的商业、质量和功能需求的组合驱动的**。它强调采用**视角和视图**来描述软件架构，采用**用例和质量属性场景**来描述需求。进一步来说，**用例描述的是功能需求，质量属性场景描述的是质量需求（或侧重于非功能需求）**。

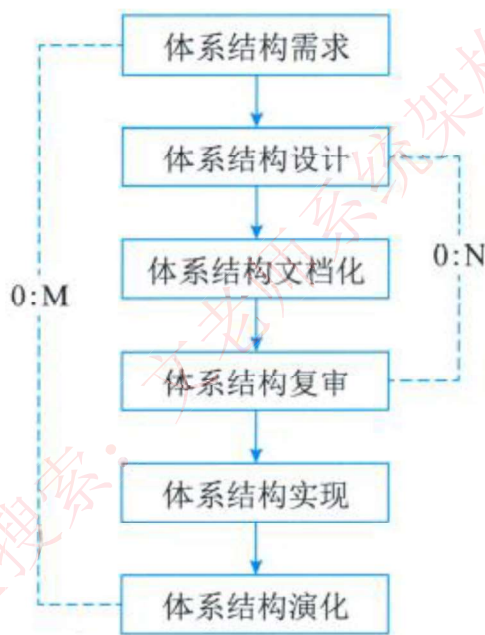
◆使用ABSD方法，**设计活动可以从项目总体功能框架明确就开始**，这意味着需求获取和分析还没有完成，就开始了软件设计。

◆ABSD方法有三个基础。**第一个基础是功能的分解**，使用已有的基于模块的内聚和耦合技术；**第二个基础是通过选择架构风格来实现质量和业务需求**；**第三个基础是软件模板的使用**，软件模板利用了一些软件系统的结构。

◆ABSD方法是一个**自顶向下，递归细化**的方法，在最顶层，系统被分解为**若干概念子系统和一个或若干个软件模板**。在第2层，概念子系统又被分解成**概念构件和一个或若干个附加软件模板**。

基于架构的软件开发

◆ ABSD模型的软件工程过程如下图所示：



基于架构的软件开发

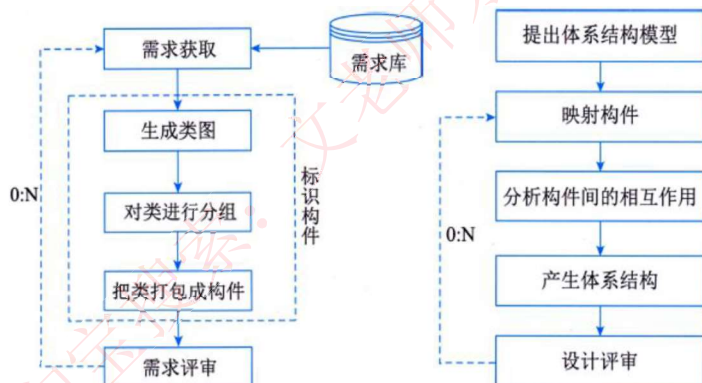
(1) 架构需求：重在掌握标识构件的三步，如下左图。

◆需求获取：体系结构需求一般来自三个方面，分别是系统的质量目标、系统的商业目标和系统开发人员的商业目标。

◆标识构件：为系统生成初始逻辑结构，标识大致的构件。包括图中三个步骤。

(2) 架构设计：将需求阶段的标识构件映射成构件，进行分析，如下右图。

(3) 架构（体系结构）文档化：主要产出两种文档，即架构（体系结构）规格说明，测试架构（体系结构）需求的质量设计说明书。文档是至关重要的，是所有人员通信的手段，关系开发的成败。

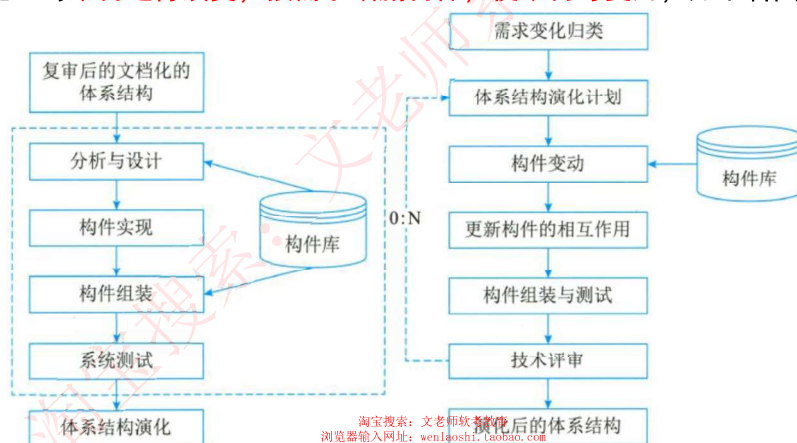


基于架构的软件开发

(4) 架构复审：安排一次由外部人员（用户代表和领域专家）参加的复审。根据架构设计，搭建一个可运行的最小化系统用于评估和测试体系架构是否满足需要。是否存在可识别的技术和协作风险。复审的目的是标识潜在的风险，及早发现体系结构设计中的缺陷和错误，包括体系结构能否满足需求、质量需求是否在设计中得到体现、层次是否清晰、构件的划分是否合理、文档表达是否明确、构件的设计是否满足功能与性能的要求等。

(5) 架构实现：用实体来显示出架构。实现构件，构件组装成系统，如下左图：

(6) 架构演化：对架构进行改变，按需求增删构件，使架构可复用，如下右图：



考试真题

下列（）不属于ADL（Architecture Description Language，架构描述语言）

A. Apriori B. Unicon C. rapide D. Acme

答案：A

ABSD方法是一个自顶向下，递归细化的方法，ABSD方法在最顶层，系统被分解为若干()和一个或者多个软件模板，架构需求一般来自三个方面，分别是系统的质量目标、系统的业务目标和（）

A:系统 B, 构件 C 模块 D 概念子系统

A,系统开发人员的业务目标 B 公司的经济目标 C 公司的业务目标 D,潜在用户的需求

答案：D A

软件架构风格

◆软件体系结构设计的一个核心目标是重复的体系结构模式，即达到体系结构级的软件重用。也就是说，在不同的软件系统中，使用同一体系结构。基于这个目标，主要任务是研究和实践软件体系结构风格和类型问题。

◆软件体系结构风格是描述某一特定应用领域中系统组织方式的惯用模式。架构风格定义一个系统家族，即一个架构定义一个词汇表和一组约束。词汇表中包含一些构件和连接件类型，而这组约束指出系统是如何将这些构件和连接件组合起来的。

◆架构风格反映了领域中众多系统所共有的结构和语义特性，并指导如何将各个模块和子系统有效地组织成一个完整的系统。对软件架构风格的研究和实践促进对设计的重用，一些经过实践证实的解决方案也可以可靠地用于解决新的问题。

软件架构风格

◆数据流风格：面向数据流，按照一定的顺序从前向后执行程序，代表的风格有批处理序列、管道-过滤器。

◆调用/返回风格：构件之间存在互相调用的关系，一般是显式的调用，代表的风格有主程序/子程序、面向对象、层次结构、客户端/服务器风格。

◆独立构件风格：构件之间是互相独立的，不存在显式的调用关系，而是通过某个事件触发、异步的方式来执行，代表的风格有进程通信、事件驱动系统（隐式调用）。

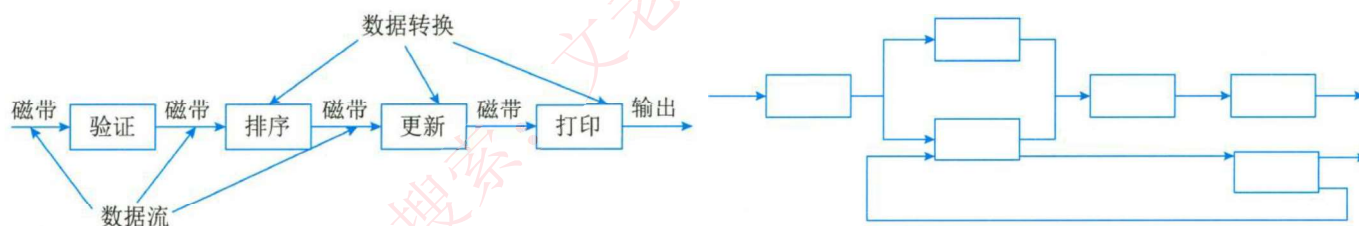
◆虚拟机风格：人为构建一个运行环境，在这个环境之上，可以解析与运行自定义的一些语言，这样来增加架构的灵活性。代表的风格有解释器、基于规则的系统。

◆以数据为中心的风格：所有的操作都是围绕建立的数据中心进行的，代表的风格有仓库风格、黑板风格。

软件架构风格

◆批处理风格：每个处理步骤是一个单独的程序，每一步必须在前一步结束后才能开始，并且数据必须是完整的，以整体的方式传递。它的基本构件是独立的应用程序，连接件是某种类型的媒介。连接件定义了相应的数据流图，表达拓扑结构。

◆管道-过滤器：把系统分解为几个序贯的处理步骤，这些步骤之间通过数据流连接，一个步骤的输出是另一个步骤的输入。每个处理步骤由一个过滤器（Filter）实现，处理步骤之间的数据传输由管道（Pipe）负责。基本构件是过滤器，连接件是数据流传输管道，将一个过滤器的输出传到另一过滤器的输入。



软件架构风格

◆调用/返回风格

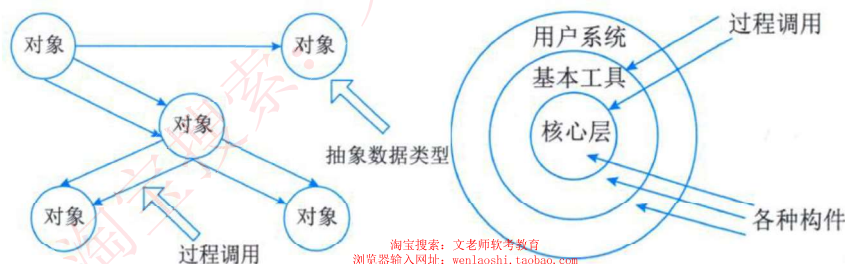
◆主程序/子程序：单线程控制，把问题划分为若干个处理步骤，构件即为主程序和子程序，子程序通常可合成为模块。过程调用作为交互机制，充当连接件的角色。

◆面向对象：构件是对象，对象是抽象数据类型的实例。连接件即使对象间交互的方式，对象是通过函数和过程的调用来交互的。

◆层次结构：构件组成一个层次结构，连接件通过决定层间如何交互的协议来定义。每层为上一层提供服务，使用下一层的服务，只能见到与自己邻接的层。修改某一层，最多影响其相邻的两层（通常只能影响上层）。

◆客户端/服务器架构风格：两层C/S 体系结构有3 个主要组成部分：数据库服务器、客户应用程序和网络。称为“胖客户机，瘦服务器”。

◆三层C / S 结构增加了一个应用服务器。整个应用逻辑驻留在应用服务器上，只有表示层存在于客户机上，故称为“瘦客户机”。应用功能分为表示层、功能层和数据层三层。



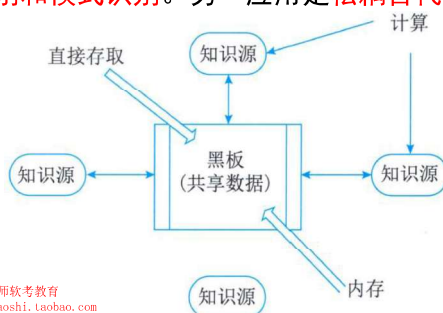
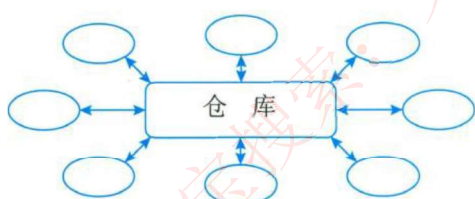
软件架构风格

◆以数据为中心的架构风格

◆仓库架构风格：**仓库是存储和维护数据的中心场所**。在仓库体系结构风格中，有两种不同的构件：**中央数据结构（说明当前数据的状态）**，以及**一组对中央数据进行操作的独立构件**，仓库与独立构件间的相互作用在系统中会有大的变化。这种风格的**连接件即为仓库与独立构件之间的交互**。

◆黑板架构风格：适用于解决**复杂的非结构化问题**，能在求解过程中综合运用多种不同**知识源**，使得问题的表达、组织和求解变得比较容易。黑板系统是一种**问题求解模型**，是组织推理步骤、控制状态数据和问题求解之领域知识的概念框架。它将**问题的解空间组织成一个或多个应用相关的分级结构**。各种应用通过不同知识表达方法、推理框架和控制机制的组合来实现。一般包括**知识源、黑板和控制**三部分。

◆黑板系统的传统应用是**信号处理领域**，如语音识别和模式识别。另一应用是**松耦合代理数据共享存取**。

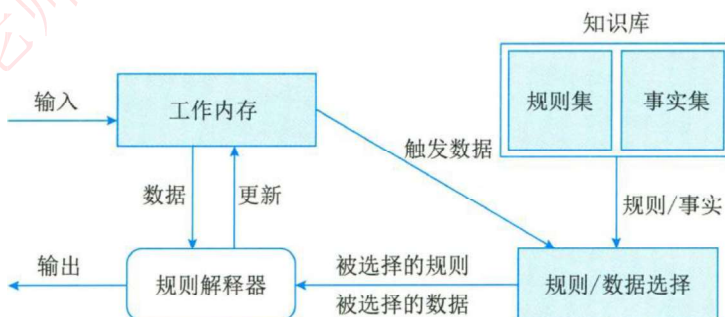
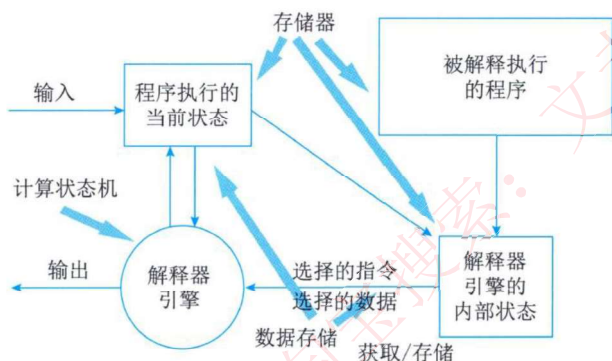


软件架构风格

◆虚拟机风格

◆解释器：通常包括一个完成解释工作的解释引擎、一个包含将被解释的代码的存储区、一个记录解释引擎当前工作状态的数据结构，以及一个记录源代码被解释执行的进度的数据结构。具有解释器风格的软件中含有一个虚拟机，可以仿真硬件的执行过程和一些关键应用，缺点是**执行效率低**。典型的例子是专家系统。

◆基于规则的系统：包括**规则集、规则解释器、规则/数据选择器和工作内存**，一般用在人工智能领域和DSS中。



软件架构风格

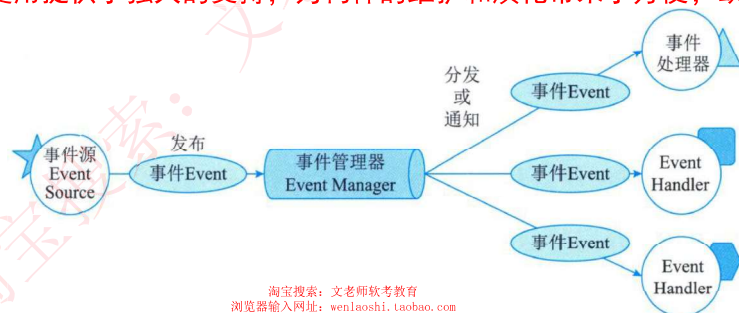
◆独立构件风格

◆进程通信：在进程通信结构体系结构风格中，构件是独立的过程，连接件是消息传递。这种风格的特点是**构件通常是命名过程**，消息传递的方式可以是**点到点、异步或同步方式及远程过程调用**等。

◆事件驱动系统（隐式调用）：事件系统风格基于事件的隐式调用风格的思想是构件**不直接调用一个过程，而是触发或广播一个或多个事件**。系统中的其他构件中的过程在一个或多个事件中注册，当一个事件被触发，系统自动调用在这个事件中注册的所有过程。

◆从架构上说，这种风格的**构件是一些模块**，这些模块既可以是一些过程，又可以是一些事件的集合。**过程可以用通用的方式调用**，也可以在系统事件中注册一些过程，当发生这些事件时，过程被调用。

◆主要优点是**为软件复用提供了强大的支持，为构件的维护和演化带来了方便**；缺点是**构件放弃了对系统计算的控制**。



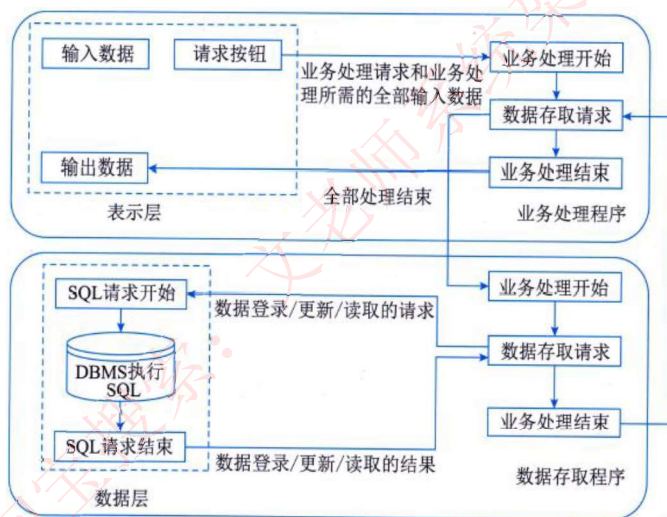
软件架构风格

架构风格名	常考关键字及实例	简介
数据流-批处理	传统编译器，每个阶段产生的	一个接一个，以整体为单位。
数据流-管道-过滤器	结果作为下一个阶段的输入， 区别在于整体。	一个接一个，前一个输出是后一个输入。

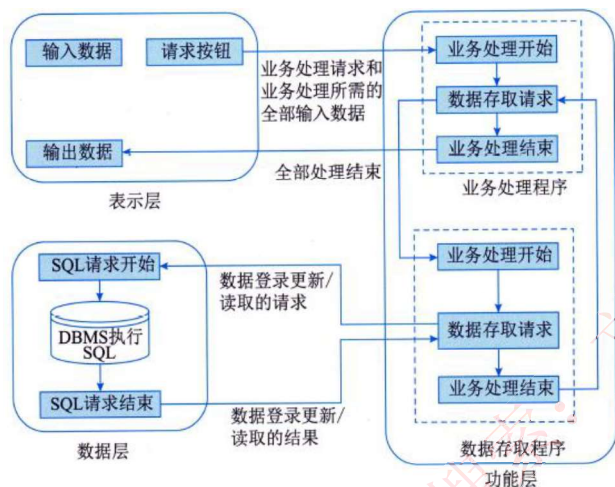
调用/返回-主程序/子程序		显示调用，主程序直接调用子程序。
调用/返回-面向对象		对象是构件，通过对象调用封装的方法和属性。
调用/返回-层次结构		分层，每层最多影响其上下两层，有调用关系。
调用/返回-客户端/服务器		两层 CS：客户端、服务器 三层 CS：表示层、功能层、数据层
独立构件-进程通信		进程间独立的消息传递，同步异步。
独立构件-事件驱动（隐式调用）	事件触发推动动作，如程序语言的语法高亮、语法错误提示	不直接调用，通过事件驱动。
虚拟机-解释器	自定义流程，按流程执行，规则随时改变，灵活定义，业务	解释自定义的规则，解释引擎、存储区、数据结构。
虚拟机-规则系统	灵活组合 机器人。	规则集、规则解释器、选择器和工作内存，用于 DSS 和人工智能、专家系统。
数据中心-仓库	现代编译器的集成开发环境	中央共享数据源，独立处理单元。
数据中心-黑板	IDE，以数据为中心。 又称为数据共享风格	语音识别、知识推理等问题复杂、解空间很大、求解过程不确定的这一类软件系统，黑板、知识源、控制。
闭环-过程控制	汽车巡航定速，空调温度调节，发动机控制	发出控制命令并接受反馈，循环往复达到平衡。

软件架构风格

◆两层C/S架构：**客户端和服务端都有处理功能，现在已经不常用**，原因有：开发成本较高、客户端程序设计复杂、信息内容和形式单一、用户界面风格不一、软件移植困难、软件维护和升级困难、新技术不能轻易应用、安全性问题、服务器端压力大难以复用。



软件架构风格



◆三层C/S架构：将**处理功能独立出来**，表示层和数据层都变得简单。**表示层在客户机上，功能层在应用服务器上，数据层在数据库服务器上**。即将两层C/S架构中的数据从服务器中独立出来了。其优点下面四点：

- ◆各层在逻辑上保持相对独立，整个系统的逻辑结构更为清晰，能提高系统和软件的可维护性和可扩展性；
- ◆允许**灵活有效的选用相应的平台和硬件系统**，具有良好的可升级性和开放性；
- ◆各层可以**并行开发**，各层也可以选择各自最适合的开发语言；
- ◆功能层有效的隔离表示层与数据层，为严格的安全管理奠定了坚实的基础，**整个系统的管理层次也更加合理和控制**。

◆三层C/S架构设计的**关键在于各层之间的通信效率**，要慎重考虑三层间的通信方法、通信频度和数据量，否则即使分配给各层的硬件能力很强，性能也不高。

软件架构风格

◆三层B/S架构：是三层C/S架构的变种，将**客户端变为用户客户端上的浏览器**，将**应用服务器变为网络上的WEB服务器**，又称为**0客户端架构**，虽然不用开发客户端，但有很多缺点：

- ◆ B/S架构缺乏对动态页面的支持能力，没有集成有效的数据库处理功能；
- ◆ 安全性难以控制；
- ◆ 在数据查询等响应速度上，要远远低于C/S架构；
- ◆ 数据提交一般以页面为单位，数据的动态交互性不强，不利于OLTP应用。

◆混合架构风格

内外有别模型：企业内部使用C/S，外部人员访问使用B/S。

查改有别模型：采用B/S查询，采用C/S修改。

富互联网应用RIA

◆弥补三层B/S架构存在的问题，**RIA是一种用户接口**，比用HTML实现的接口更加健壮，且有可视化内容，本质还是网站模式，其优点如下：

- ◆ RIA结合了C/S架构反应速度快、交互性强的优点与B/S架构传播范围广及容易传播的特性；
- ◆ RIA简化并改进了B/S架构的用户交互；
- ◆ 数据能够被缓存在客户端，从而可以实现一个比基于HTML的响应速度更快且数据往返于服务器的次数更少的用户界面。
- ◆ 本质还是0客户端，借助于**高速网速实现必要插件在本地的快速缓存**，增强页面对动态页面的支持能力，典型的如小程序。

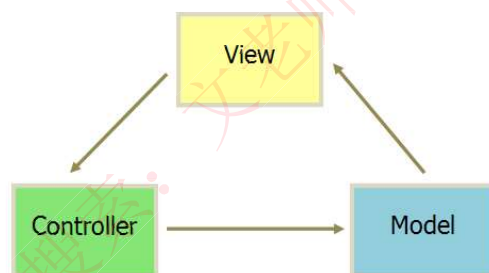
软件架构风格

MVC架构

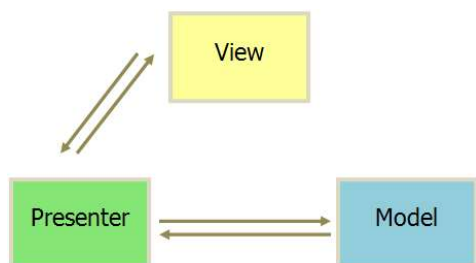
(1) 控制器（Controller）：是应用程序中**处理用户交互的部分**。通常控制器负责从视图读取数据，控制用户输入，并向模型发送数据。

(2) 模型（Model）：是应用程序中用于**处理应用程序数据逻辑的部分**。通常模型对象负责在数据库中存取数据。模型表示业务数据和业务逻辑。

(3) 视图（View）：是应用程序中**处理数据显示的部分**。通常视图是依据模型数据创建的。是用户看到并与之交互的界面。视图向用户显示相关的数据，并能接收用户的输入数据，但是它并不进行任何实际的业务处理：



软件架构风格

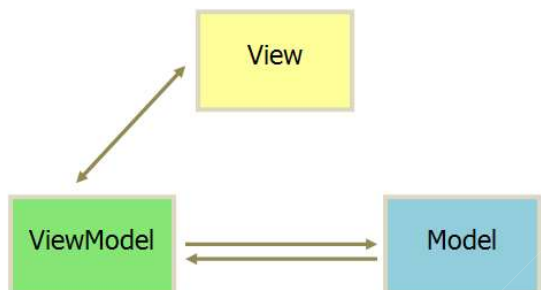


MVP: MVP是把MVC中的Controller换成了Presenter（呈现），目的就是完全切断View跟Model之间的联系，由Presenter充当桥梁，做到View-Model之间通信的完全隔离。

MVP特点：

- 1、M、V、P之间双向通信。
- 2、View 与 Model 不通信，都通过 Presenter 传递。Presenter完全把Model和View进行了分离，主要的程序逻辑在Presenter里实现。
- 3、View 非常薄，不部署任何业务逻辑，称为”被动视图”（Passive View），即没有任何主动性，而 Presenter非常厚，所有逻辑都部署在那里。
- 4、Presenter与具体的View是没有直接关联的，而是通过定义好的接口进行交互，从而使得在变更View时候可以保持Presenter的不变，这样就可以重用。

软件架构风格



MVVM: MVVM模式和MVC模式类似，主要目的是分离视图（View）和模型（Model），有几点优点：

- 1、低耦合，视图（View）可以独立于Model变化和修改，一个ViewModel可以绑定到不同的”View”上，当View变化的时候Model可以不变，当Model变化的时候View也可以不变。
- 2、可重用性，可以把一些视图逻辑放在一个ViewModel里面，让很多view重用这段视图逻辑。
- 3、独立开发，开发人员可以专注于业务逻辑和数据的开发（ViewModel），设计人员可以专注于页面设计。
- 4、可测试，界面向来是比较难于测试的，而现在测试可以针对ViewModel来写。

考试真题

在黑板架构风格中，数据处理单元是（）

- A.中央数据结构 B.知识源 C.黑板 D.控制器

答案：A

以下关于架构风格的说法中，不正确的是（）

- A.虚拟机风格自定义了一套规则供使用者使用，使用者基于这个规则来开发构件
B.事件驱动系统中，一个事件的发生通常会触发或影响其他事件
C.仓库风格以数据为中心，所有的操作都是围绕建立的数据中心进行的
D.黑板系统包括知识源、黑板和控制三部分

答案：B

以下关于层次式架构风格的说法中正确的是（）

- A.两层CS架构分为表示层和数据层，是一种瘦客户端架构
B.三层CS架构一般分为表示层、数据抽象层和数据层
C.物联网的典型架构分为感知层、网络层和应用层
D.BS架构的优点在于支持动态页面，性能相对于CS架构更好

答案：C

考试真题

某公司拟开发一个VIP管理系统，系统需要根据不同商场活动，不定期更新VIP会员的审核标准和VIP折扣系统。针对上述需求，采用（）架构风格最为合适。

- A.规则系统 B.过程控制 C.分层 D.管道-过滤器

答案：A

某公司拟为某种新型可编程机器人开发相应的编译器。该编译过程包括词法分析、语法分析、语义分析和代码生成四个阶段，每个阶段产生的结果作为下一个阶段的输入，且需独立存储。针对上述描述，该集成开发环境应采用（）架构风格最为合适。

- A.管道-过滤器 B.数据仓储 C.主程序-子程序 D.解释器

答案：A

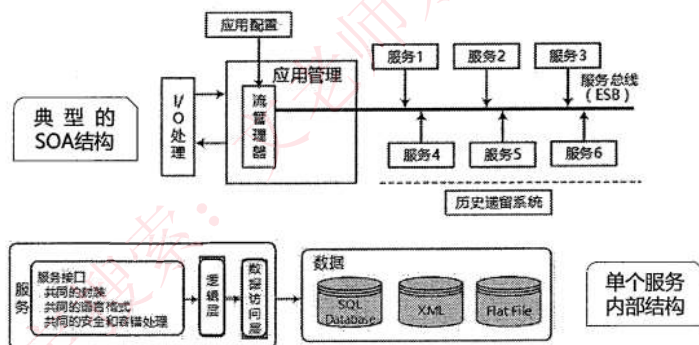
某企业内部现有的主要业务功能已封装成为Web服务。为了拓展业务范围，需要将现有的业务功能进行多种组合，形成新的业务功能。针对业务灵活组合这一要求，采用（）架构风格最为合适。

- A.规则系统 B.面向对象 C.黑板 D.解释器

答案：D

面向服务的架构

- ◆SOA是一种**粗粒度、松耦合服务架构**，服务之间通过简单、精确定义接口进行通信，不涉及底层编程接口和通信模型。
- ◆在SOA中，**服务是一种为了满足某项业务需求的操作、规则等的逻辑组合**，它包含一系列有序活动的交互，为实现用户目标提供支持。
- ◆SOA并不仅仅是一种开发方法，还具有管理上的优点，**管理员可直接管理开发人员所构建的相同服务**。多个服务**通过企业服务总线提出服务请求**，由应用管理来进行处理，如下：



面向服务的架构

◆实施SOA的关键目标是**实现企业IT资产重用的最大化**，在实施SOA过程中要牢记以下特征：**可从企业外部访问、随时可用（服务请求能被及时响应）、粗粒度接口（粗粒度提供一项特定的业务功能，而细粒度服务代表了技术构件方法）、服务分级、松散耦合（服务提供者和服务使用者分离）、可重用的服务及服务接口设计管理、标准化的接口（WSDL、SOAP、XML是核心）、支持各种消息模式、精确定义的服务接口。**

◆从基于对象到基于构件再到基于服务，**架构越来越松散耦合，粒度越来越粗，接口越来越标准。**

◆基于服务的构件与传统构件的区别有四点：

服务构件**粗粒度**，传统构件细粒度居多；

服务构件的**接口是标准的**，主要是**WSDL接口**，而传统构件常以具体API形式出现；

服务构件的**实现与语言是无关的**，而传统构件常绑定某种特定的语言；

服务构件可以**通过构件容器提供QoS的服务**，而传统构件完全由程序代码直接控制。

面向服务的架构

◆ Web服务最基本的协议包括**UDDI**、**WSDL**和**SOAP**, 通过它们, 可以提供直接而又简单的Web Service 支持, 如图所示。

1.**UDDI (统一描述、发现和集成协议)**计划是一个广泛的、开放的行业计划, 它使得**商业实体能够彼此发现**; 定义它们怎样在Internet上互相作用, 并在一个全球的**注册体系架构中共享信息**。



面向服务的架构

2.**WSDL(Web 服务描述语言)**,是一个用来**描述Web服务和说明如何与Web服务通信的XML语言**。可描述三个基本属性:

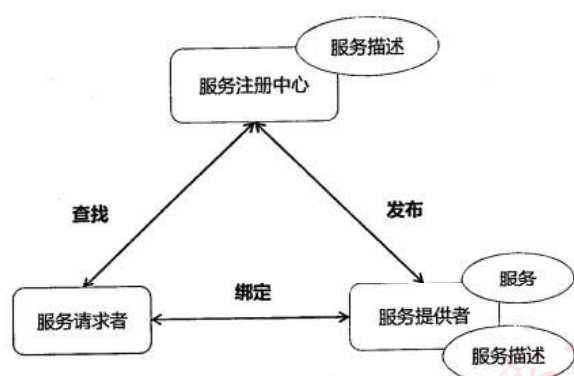
- (1) 服务做什么——服务所提供的操作(方法)。
- (2) 如何访问服务——和服务交互的数据格式以及必要协议。
- (3) 服务位于何处协议相关的地址, 如URL。

◆ WSDL文档以**端口集合**的形式来描述Web 服务, WSDL 服务描述包含对**一组操作和消息的一个抽象定义**, 绑定到这些操作和消息的一个具体协议, 和这个绑定的一个网络端点规范。WSDL文档被分为两种类型:**服务接口和服务实现**。

3.**SOAP是在分散或分布式的环境中交换信息的简单的协议**, 是一个**基于XML**的协议。它包括4个部分:**SOAP封装**, 定义了一个描述消息中的内容是什么, 是谁发送的, 谁应当接收并处理它以及如何处理它们的框架; **SOAP编码规则**, 用于表示应用程序需要使用的数据类型实例; **SOAP RPC**表示是远程过程调用和应答的协定; **SOAP绑定**是使用底层协议交换信息。

◆ SOAP 的两个主要设计目标是**简单性和可扩展性**, 这就意味着有一些传统消息系统或分布式对象系统中的某些性质将不是SOAP 规范的一部分。

面向服务的架构



◆ WEB Service

◆ **服务提供者**、**服务注册中心**（中介，提供交易平台，可有可无）、**服务请求者**。服务提供者将服务描述发布到服务注册中心，供服务请求者查找，查找到后，服务请求者将绑定查找结果。

◆ 分为六层：**底层传输层**（负责底层消息的传输，采用HTTP、JMS、SMTP协议），**服务通信协议层**（描述并定义服务间通信的技术标准，采用SOAP和REST协议），**服务描述层**（采用WSDL协议），**服务层**（对企业应用系统进行包装，通过WSDL定义的标准进行调用），**业务流程层**（支持服务发现、服务调用和点到点的服务调用，采用WSDPEL标准），**服务注册层**（采用UDDI协议）。

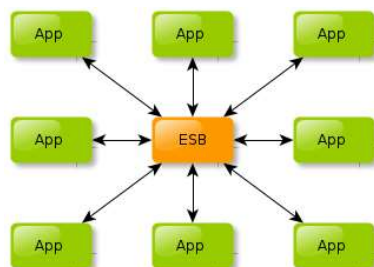
◆ 服务注册表

(1) **服务注册**：应用开发者（服务提供者）在注册表中公布服务的功能。

(2) **服务位置**：服务使用者（服务应用开发者），帮助他们查询注册服务，寻找符合自身要求的服务。

(3) **服务绑定**：服务使用者利用检索到的服务接口来编写代码，所编写的代码将与注册的服务绑定，调用注册的服务，以及与其它实现互动。

面向服务的架构



◆ **企业服务总线ESB**：简单来说是一根管道，用来连接各个服务节点。ESB的存在是为了**集成基于不同协议的不同服务**，ESB 做了消息的转化、解释以及路由的工作，以此来让不同的服务互联互通。

◆ 包括：**客户端**（服务请求者）、**基础架构服务**（中间件）、**核心集成服务**（提供服务）。

◆ ESB功能：

提供**位置透明性的消息路由和寻址服务**；

提供**服务注册和命名**的管理功能；

支持多种的**消息传递范型**；

支持多种可以广泛使用的**传输协议**；

支持多种**数据格式及其相互转换**；

提供**日志和监控**功能。

软件架构复用

◆软件产品线是指一组软件密集型系统，它们共享一个公共的、可管理的特性集，满足某个特定市场或任务的具体需要，是以规定的方式用公共的核心资产集成开发出来的。即围绕核心资产库进行管理、复用、集成新的系统。核心资产库包括软件架构及其可剪裁的元素，更广泛地，它还包括设计方案及其文档、用户手册、项目管理的历史记录(如预算和进度)、软件测试计划和测试用例。

◆软件复用是一种系统化的软件开发过程，通过识别、开发、分类、获取和修改软件实体，以便在不同的软件开发过程中重复的使用它们。

◆软件架构复用的类型包括机会复用和系统复用。机会复用是指开发过程中，只要发现有可复用的资产，就对其进行复用。系统复用是指在开发之前，就要进行规划，以决定哪些需要复用。

◆可复用的资产包括：需求、架构设计、元素、建模与分析、测试、项目规划、过程方法和工具、人员、样本系统、缺陷消除。

◆复用的基本过程主要包括3个阶段：

1. 首先构造 / 获取可复用的软件资产。
2. 管理可复用资产。最重要的是构件库，构件库应提供的主要功能包括构件的存储、管理、检索以及库的浏览与维护等，以及支持使用者有效地、准确地发现所需的可复用构件。存在两个关键问题，一是构件分类；二是构件检索。
3. 使用可复用资产。在最后阶段，通过获取需求，检索复用资产库，获取可复用资产，并定制这些可复用资产：修改、扩展、配置等，最后将它们组装与集成，形成最终系统。

考试真题

下面（）协议定义了服务请求者和提供者之间的数据传输规范

- A.UDDI B.SOAP C.WSDL D.XML

答案：B

以下关于SOAP描述错误的是（）

- A.SOAP是一个基于XML的协议
B.SOAP中的信封和XML编码定义在相同命名空间
C.SOAP封装,定义了一个描述消息中的内容是什么，是谁发送的，谁应当接收并处理它以及如何处理它们的框架
D.SOAP RPC表示是远程过程调用和应答的协定

答案：B

以下关于架构复用的说法中正确的是（）

- A.架构复用的基本过程主要包括3个阶段：首先构造 / 获取可复用的软件资产，其次管理这些资产（构件库），最后开发特定的可复用资产。
B.管理可复用资产涉及对可复用资产的分析
C.管理可复用资产过程涉及两个关键问题，一是构件获取，二是构件检索
D.软件架构复用的类型包括机会复用和构件复用

答案：B

特定领域软件架构

◆DSSA就是**专用于一类特定类型的任务(领域)**的、在整个领域中能有效地使用的、为成功构造应用系统限定了标准的组合结构的**软件构件的集合**。

◆ DSSA就是一个**特定的问题领域**中支持一组应用的领域模型、参考需求、参考架构等组成的开发基础，其目标就是**支持在一个特定领域中多个应用的生成**。

◆DSSA 的必备特征如下。

- (1) 一个**严格定义的问题域和问题解域**。
- (2) 具有**普遍性**，使其可以用于领域中某个特定应用的开发。
- (3) 对整个领域的构件组织模型的恰当**抽象**。
- (4) 具备该领域**固定的、典型的在开发过程中可重用元素**。

◆有两种理解DSSA种领域的含义如下：

- (1) 垂直域：在**一个特定领域中的通用的软件架构，是一个完整的架构**。
- (2) 水平域：在**多个不同的特定领域之间的相同的部分的小工具**（如购物和教育都有收费系统，收费系统即是水平域）。

特定领域软件架构

◆DSSA的三个基本活动

领域分析：这个阶段的主要目标是**获得领域模型（领域需求）**。首先要进行一些准备性的活动，包括定义领域的边界，从而明确分析的对象。**识别信息源**，即整个领域工程过程中信息的来源，可能的信息源包括现存系统、技术文献、问题域和系统开发的专家、用户调查和市场分析、领域演化的历史记录等，**在此基础上就可以分析领域中系统的需求，确定哪些需求是领域中的系统广泛共享的，从而建立领域模型**。

领域设计：这个阶段的目标是**获得DSSA**。DSSA描述在领域模型中表示的需求的**解决方案**，它不是单个系统的表示，而是能够**适应领域中多个系统的需求的一个高层次的设计**。建立了领域模型之后，就可以派生出满足这些被建模的领域需求DSSA。

领域实现：这个阶段的主要目标是依据领域模型和DSSA**开发和组织可重用信息**。这些可重用信息可能是从现有系统中提取得到，也可能需要通过新的开发得到。

◆以上过程是一个**反复的、逐渐求精的过程**。在实施领域工程的每个阶段中，**都可能返回到以前的步骤，对以前的步骤得到的结果进行修改和完善**，再回到当前步骤，在新的基础上进行本阶段的活动。

特定领域软件架构

◆参与DSSA的四种角色人员

领域专家：包括该领域中系统的有经验的用户、从事该领域中系统的需求分析、设计、实现以及项目管理的有经验的软件工程师等。提供关于领域中系统的需求规约和实现的知识，帮助组织规范的、一致的领域字典，帮助选择样本系统作为领域工程的依据，复审领域模型、DSSA等领域工程产品，等等。

领域分析人员：由具有知识工程背景的系统分析员来担任。控制整个领域分析过程，进行知识获取，将获取的知识组织到领域模型中。

领域设计人员：由有经验的软件设计人员来担任。根据领域模型和现有系统开发出DSSA，并对DSSA的准确性和一致性进行验证。

领域实现人员：由有经验的程序设计人员来担任。根据领域模型和DSSA，开发构件。

特定领域软件架构

◆DSSA的建立过程分为5个阶段，每个阶段可以进一步划分为一些步骤或子阶段。每个阶段包括一组需要回答的问题，一组需要的输入，一组将产生的输出和验证标准。完成本过程可能需要对每个阶段经历几遍，每次增加更多的细节，类似螺旋模型。

(1) **定义领域范围**：本阶段的重点是确定什么在感兴趣的领域中以及本过程到何时结束。输出是领域中的应用要满足用户一系列的需求。

(2) **定义领域特定的元素**：本阶段的目标是编译领域字典和领域术语的同义词词典。归纳领域中的术语，识别出领域中相同和不相同的元素。

(3) **定义领域特定的设计和实现需求的约束**：识别领域中的所有约束，这些约束对领域的设计和实现会造成什么后果。

(4) **定义领域模型和架构**：产生一般的架构，并描述其构件说明。

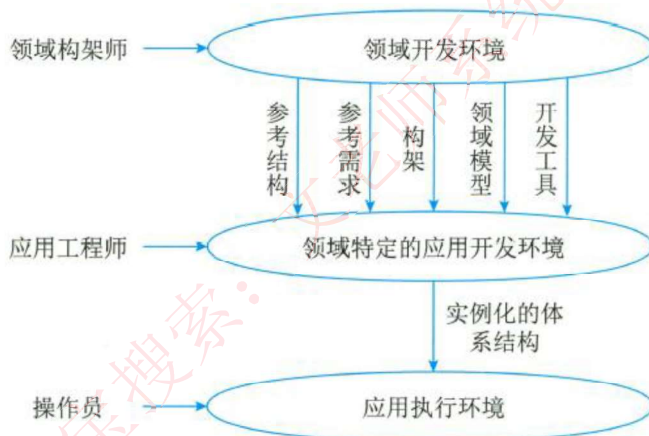
(5) **产生、搜集可复用的产品单元**：为DSSA增加复用构件，使可用于新的系统。

◆ DSSA 的建立过程是并发的、递归的和反复进行的。该过程的目的是将用户的需求映射为基于实现限制集合的软件需求，这些需求定义了DSSA。在此之前的领域工程和领域分析过程并没有对系统的功能性需求和实现限制进行区分，而是统称为“需求”。下图DSSA 的一个三层次系统模型。

特定领域软件架构

三层次模型

- ◆ **领域开发环境**：领域架构师决定核心架构，产出参考结构、参考需求、架构、领域模型、开发工具。
- ◆ **领域特定的应用开发环境**：应用工程师根据具体环境来将核心架构实例化。
- ◆ **应用执行环境**：操作员实现实例化后的架构。



考试真题

以下关于建立DSSA的过程的说法中，（）是错误的

- A. DSSA的建立过程分为五个阶段，每个阶段可以进一步划分为一些步骤或子阶段
- B. 每个阶段包括一组需要回答的问题，一组需要的输入，一组将产生的输出和验证标准
- C. 完成本过程可能需要对每个阶段经历几遍，每次增加更多的细节
- D. 本过程是并发的、递归的、反复的，或者说，它是原型模型

答案：D

特定领域软件架构（Domain Specific Software Architecture, DSSA）以一个特定问题领域为对象，形成由领域参考模型，参考需求，（）等组成的开发基础架构，支持一个特定领域中多个应用的生成。DSSA的基本活动包括领域分析、领域设计和领域实现。其中领域分析的主要目的是获得（），从而描述领域中系统之间共同的需求，即领域需求；领域设计的主要目标是获得（），从而描述领域模型中表示需求的解决方案；领域实现的主要目标是开发和组织可重用信息，并实现基础软件架构。

- | | | | |
|---------------|---------------|---------|---------|
| A. 参考设计 | B. 参考规约 | C. 参考架构 | D. 参考实现 |
| A. 领域边界 | B. 领域信息 | C. 领域对象 | D. 领域模型 |
| A. 特点领域软件需求 | B. 特定领域软件架构 | | |
| C. 特定领域软件设计模型 | D. 特定领域软件重用模型 | | |
- 答案：C D B

软件系统的质量属性

- ◆软件系统的质量就是“软件系统与明确地和隐含地定义的需求相一致的程度”。
- ◆从管理角度对软件系统质量进行度量，可将影响软件质量的主要因素划分为6种维度特性：**功能性、可靠性、易用性、效率、维护性与可移植性**。其中
 - ◆**功能性**包括适合性、准确性、互操作性、依从性、安全性；
 - ◆**可靠性**包括容错性、易恢复性、成熟性；
 - ◆**易用性**包括易学性、易理解性、易操作性；
 - ◆**效率**包括资源特性和时间特性；
 - ◆**维护性**包括可测试性、可修改性、稳定性和易分析性；
 - ◆**可移植性**包括适应性、易安装性、一致性和可替换性。

软件系统的质量属性

- ◆可以将软件系统的质量属性分为**开发期质量属性**和**运行期质量属性**2个部分。
 - 1.开发期质量属性主要指在**软件开发阶段**所关注的质量属性，主要包含6个方面。
 - (1) 易理解性：指设计被开发人员理解的难易程度。
 - (2) 可扩展性：软件因适应新需求或需求变化而增加新功能的能力，也称为灵活性。
 - (3) 可重用性：指重用软件系统或某一部分的难易程度。
 - (4) 可测试性：对软件测试以证明其满足需求规范的难易程度。
 - (5) 可维护性：当需要修改缺陷、增加功能、提高质量属性时，识别修改点并实施修改的难易程度。
 - (6) 可移植性：将软件系统从一个运行环境转移到另一个不同的运行环境的难易程度。
 - 2.运行期质量属性主要指在**软件运行阶段**所关注的质量属性，主要包含7个方面。
 - (1) 性能：性能是指软件系统及时提供相应服务的能力，如速度、吞吐量和容量等的要求。
 - (2) 安全性：指软件系统同时兼顾向合法用户提供服务，以及阻止非授权使用的能力。
 - (3) 可伸缩性：指当用户数和数据量增加时，软件系统维持高服务质量的能力。例如，通过增加服务器来提高能力。
 - (4) 互操作性：指本软件系统与其他系统交换数据和相互调用服务的难易程度。
 - (5) 可靠性：软件系统在一定的时间内持续无故障运行的能力。
 - (6) 可用性：指系统在一定时间内正常工作的时间所占的比例。可用性会受到系统错误，恶意攻击，高负载等问题的影响。
 - (7) 鲁棒性：是指软件系统在非正常情况（如用户进行了非法操作、相关的软硬件系统发生了故障等）下仍能够正常运行的能力，也称**健壮性或容错性**。

软件系统的质量属性

◆面向架构评估的质量属性

1、性能：指**系统的响应能力**，即要经过多长时间才能对某个事件做出响应，或者在某段时间内系统所能处理的事件的个数。如**响应时间、吞吐量**。

设计策略：**优先级队列、增加计算资源、减少计算开销、引入并发机制、采用资源调度等**。

2、可靠性：是软件系统在应用或系统错误面前，在**意外或错误使用的情况下维持软件系统的功能特性**的基本能力。如**MTTF、MTBF、MTTR**。包括**容错和健壮性**。

设计策略：**心跳、Ping/Echo、冗余、选举**。

3、可用性：是系统**能够正常运行的时间比例**，经常用两次故障之间的时间长度或在出现故障时系统能够恢复正常的速度来表示。如**故障间隔时间**。

设计策略：**心跳、Ping/Echo、冗余、选举**。

4、安全性：是指**系统在向合法用户提供服务的同时能够阻止非授权用户使用的企图或拒绝服务的能力**。如**保密性、完整性、不可抵赖性、可控性**。

设计策略：**入侵检测、用户认证、用户授权、追踪审计**。

软件系统的质量属性

5、可修改性：指能够快速的**以较高的性能价格比对系统进行变更**的能力。通常以某些具体的变更为基准，通过考察这些变更的代价衡量。包括**可维护性、可扩展性、结构重组、可移植性**。

设计策略：**接口-实现分类、抽象、信息隐藏**。

6、功能性：是**系统所能完成所期望的工作的能力**。一项任务的完成需要系统中许多或大多数构件的相互协作。

7、可变性：指**体系结构经扩充或变更而成为新体系结构的能力**。这种新体系结构应该符合预先定义的规则，在某些具体方面不同于原有的体系结构。当要将某个体系结构作为一系列相关产品的基础时，可变性是很重要的。

8、互操作性：作为系统**组成部分的软件不是独立存在的，经常与其他系统或自身环境相互作用**。为了支持互操作性，软件体系结构必须为外部可视的功能特性和数据结构提供精心设计的软件入口。程序和用其他编程语言编写的软件系统的交互作用就是互操作性的问题，也影响应用的软件体系结构。

软件系统的质量属性

◆质量属性场景是一种面向特定质量属性的需求。它由6部分组成：

- 刺激源（Source）：这是某个生成该刺激的实体（人、计算机系统或者任何其他刺激器）。
- 刺激（Stimulus）：该刺激是当刺激到达系统时需要考虑的条件。
- 环境（Environment）：该刺激在某些条件内发生。当激励发生时，系统可能处于过载、运行或者其他情况。
- 制品（Artifact）：某个制品被激励。这可能是整个系统，也可能是系统的一部分。
- 响应（Response）：该响应是在激励到达后所采取的行动。
- 响应度量（Measurement）：当响应发生时，应当能够以某种方式对其进行度量，以对需求进行测试。

软件系统的质量属性

◆可用性质量属性场景：关注的方面包括系统故障发生的频率、出现故障时会发生什么情况、允许系统有多长是非正常运行、什么时候可以安全地出现故障、如何防止故障的发生以及发生故障时要求进行哪种通知。

场景要素	可能的情况
刺激源	系统内部、系统外部
刺激	疏忽、错误、崩溃、时间
环境	正常操作、降级模式
制品	系统处理器、通信信道、持久存储器、进程
响应	系统应该检测事件、并进行如下一个或多个活动： 将其记录下来通知适当的各方，包括用户和其他系统；根据已定义的规则禁止导致错误或故障的事件源 在一段预先指定的时间间隔内不可用，其中，时间间隔取决于系统的关键程度在正常或降级模式下运行
响应度量	系统必须可用的时间间隔 可用时间 系统可以在降级模式下运行的时间间隔 故障修复时间

软件系统的质量属性

◆可修改性质量属性场景：主要关注系统在**改变功能、质量属性时需要付出的成本和难度**，可修改性质量属性场景可能发生在系统设计、编译、构建、运行等多种情况和环境下。

场景要素	可能的情况
刺激源	最终用户、开发人员、系统管理员
刺激	希望增加、删除、修改、改变功能、质量属性、容量等
环境	系统设计时、编译时、构建时、运行时
制品	系统用户界面、平台、环境或与目标系统交互的系统
响应	查找架构中需要修改的位置，进行修改且不会影响其他功能，对所做的修改进行测试，部署所做的修改
响应度量	根据所影响元素的数量度量的成本、努力、资金；该修改对其他功能或质量属性所造成影响的程度

软件系统的质量属性

◆性能质量属性场景：主要关注系统的**响应速度**，可以通过**效率、响应时间、吞吐量、负载**来客观评价性能的好坏。

场景要素	可能的情况
刺激源	用户请求，其他系统触发等
刺激	定期事件到达、随机事件到达、偶然事件到达
环境	正常模式、超载（Overload）模式
制品	系统
响应	处理刺激、改变服务级别
响应度量	等待时间、期限、吞吐量、抖动、缺失率、数据丢失率

软件系统的质量属性

◆可测试性质量属性场景：主要关注系统测试过程中的效率，发现系统缺陷或故障的难易程度等。

场景要素	可能的情况
刺激源	开发人员、增量开发人员、系统验证人员、客户验收测试人员、系统用户
刺激	已完成的分析、架构、设计、类和子系统集成；所交付的系统
环境	设计时、开发时、编译时、部署时
制品	设计、代码段、完整的应用
响应	提供对状态值的访问，提供所计算的值，准备测试环境
响应度量	已执行的可执行语句的百分比 如果存在缺陷出现故障的概率 执行测试的时间 测试中最长依赖的长度 准备测试环境的时间

软件系统的质量属性

◆易用性质量属性场景：主要关注用户在使用系统时的容易程度，包括系统的学习曲线、完成操作的效率、对系统使用过程的满意程度等。

场景要素	可能的情况
刺激源	最终用户
刺激	想要学习系统特性、有效使用系统、使错误的影响最低、适配系统、对系统满意
环境	系统运行时或配置时
制品	系统
响应	(1) 系统提供以下一个或多个响应来支持“学习系统特性”： 帮助系统与环境联系紧密；界面为用户所熟悉；在不熟悉的环境中，界面是可以使用的 (2) 系统提供以下一个或多个响应来支持“有效使用系统”： 数据和（或）命令的聚合；已输入的数据和（或）命令的重用；支持在界面中的有效导航；具有一致操作的不同视图；全面搜索；多个同时进行的活动 (3) 系统提供以下一个或多个响应来“使错误的影响最低”： 撤销；取消；从系统故障中恢复；识别并纠正用户错误；检索忘记的密码；验证系统资源 (4) 系统提供以下一个或多个响应来“适配系统”： 定制能力；国际化 (5) 系统提供以下一个或多个响应来使客户“对系统的满意”： 显示系统状态；与客户的节奏合拍
响应度量	任务时间、错误数量、解决问题的数量、用户满意度、用户知识的获得、成功操作在总操作中所占的比例、损失的时间、丢失的数据量

软件系统的质量属性

◆安全性质量属性场景: 主要关注系统在安全性方面的要素, 衡量系统在向合法用户提供服务的同时, 阻止非授权用户使用的能力。

场景要素	可能的情况
刺激源	正确识别、非正确识别身份未知的来自内部 / 外部的个人或系统; 经过了授权 / 未授权它访问了有限的资源 / 大量资源
刺激	试图显示数据, 改变 / 删除数据, 访问系统服务, 降低系统服务的可用性
环境	在线或离线、联网或断网、连接有防火墙或者直接连到了网络
制品	系统服务、系统中的数据
响应	对用户身份进行认证; 隐藏用户的身份; 阻止对数据或服务的访问; 允许访问数据或服务; 授予或收回对访问数据或服务的许可; 根据身份记录访问、修改或试图访问、修改数据、服务; 以一种不可读的格式存储数据; 识别无法解释的对服务的高需求; 通知用户或另外一个系统, 并限制服务的可用性
响应度量	用成功的概率表示, 避开安全防范措施所需要的时间、努力、资源; 检测到攻击的可能性; 确定攻击或访问、修改数据或服务的个人的可能性; 在拒绝服务攻击的情况下仍然获得服务的百分比; 恢复数据、服务; 被破坏的数据、服务和 (或) 被拒绝的合法访问的范围

考试真题

下列 () 不属于可靠性的子特性

A.容错性 B.成熟性 C.可恢复性 D.稳定性

答案: D

质量属性场景的六要素中, () 是激励到达后所采取的行动

A. 刺激源 B. 响应 C. 制品 D. 环境

答案: B

某公司欲开发一个在线交易网站, 在架构设计阶段, 公司的架构师识别出 3 个核心质量属性场景。其中"网站正常运行时, 用户发起的交易请求应该在 3 秒内完成" 主要与 (58) 质量属性相关, 通常可采用 (59) 架构策略实现该属性;"在线交易主站宕机后, 能够在 3 秒内自动切换至备用站点并恢复正常运行"主要与 (60) 质量属性相关, 通常可采用 (61) 架构策略实现该属性;"系统应该具备一定的安全保护措施, 从而能够抵挡恶意的入侵破坏行为, 并对所有针对网站的攻击行为进行报警和记录"主要与 (62) 质量属性相关, 通常可采用 (63) 架构策略实现该属性。

- | | | | |
|---------------|-------------|---------|-----------|
| (58) A. 可用性 | B.性能 | C. 易用性 | D. 可修改性 |
| (59) A. 抽象接口 | B.信息隐藏 | C. 主动冗余 | D. 资源调度 |
| (60) A. 可测试性 | B.易用性 | C. 可用性 | D. 互操作性 |
| (61) A. 记录/回放 | B. 操作串行化 | C.心跳 | D. 增加计算资源 |
| (62) A. 可用性 | B. 安全性 | C. 可测试性 | D. 可修改性 |
| (63) A. 追踪审计 | B.Ping/Echo | C.选举 | D. 维护现有接口 |

答案: (58) B (59) D (60) C (61) C (62) B (63) A

软件架构评估

- ◆系统架构评估是在对架构分析、评估的基础上，对架构策略的选取进行决策。
- ◆软件架构评估在架构设计之后，系统设计之前，因此与设计、实现、测试都没有关系。评估的目的是为了评估所采用的架构是否能解决软件系统需求，但不是单纯的确定是否满足需求。
- ◆敏感点：是指为了实现某种特定的质量属性，一个或多个构件所具有的特性。
- ◆权衡点：是影响多个质量属性的特性，是多个质量属性的敏感点。
- ◆风险承担者或者称为利益相关人。系统的架构涉及很多人的利益，这些人人都对架构施加各种影响，以保证自己的目标能够实现。
- ◆场景：在进行架构评估时，一般首先要精确地得出具体的质量目标，并以之作为判定该架构优劣的标准。为得出这些目标而采用的机制称之为场景。场景是从风险承担者的角度对与系统的交互的简短描述。在架构评估中，一般采用刺激、环境和响应三方面来对场景进行描述。
- ◆风险点与非风险点不是以标准专业术语形式出现的，只是一个常规概念，即可能引起风险的因素，可称为风险点。某个做法如果有隐患，有可能导致一些问题，则为风险点；而如果某件事是可行的可接受的，则为非风险点。

软件架构评估

- ◆三种常用的评估方式
 - (1) 基于调查问卷或检查表的方法。该方法的关键是要设计好问卷或检查表，充分利用系统相关人员的经验和知识，获得对架构的评估。该方法的缺点是在很大程度上依赖于评估人员的主观推断。
 - (2) 基于场景的评估方法。应用在架构权衡分析法和软件架构分析方法中。它是通过分析软件架构对场景（也就是对系统的使用或修改活动）的支持程度，从而判断该架构对这一场景所代表的质量需求的满足程度。
 - (3) 基于度量的评估方法。它是建立在软件架构度量的基础上的，涉及3个基本活动，首先需要建立质量属性和度量之间的映射原则，然后从软件架构文档中获取度量信息，最后根据映射原则分析推导出系统的质量属性。

评估方式	调查问卷或检查表		场景	度量
	调查问卷	检查表		
通用性	通用	特定领域	特定系统	通用或特定领域
评估者对架构的了解程度	粗略了解	无限制	中等了解	精确了解
实施阶段	早	中	中	中
客观性	主观	主观	较主观	较客观

软件架构评估

◆软件架构分析方法SAAM

SAAM是一种**非功能质量属性**的架构分析方法，是**最早形成文档**并得到广泛应用的软件架构分析方法。

(1) 特定目标。SAAM的目标是**对描述应用程序属性的文档**，验证基本的架构假设和原则。

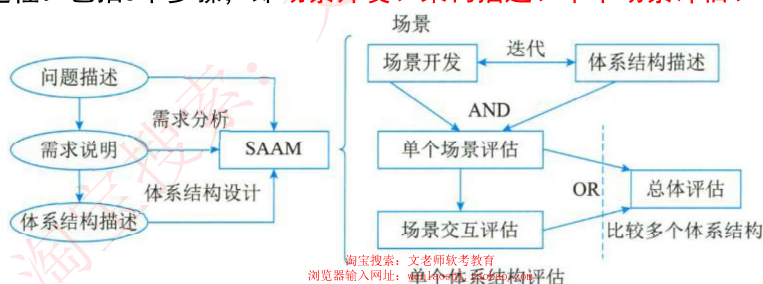
(2) 评估技术。SAAM 所使用的评估技术是**场景技术**。

(3) 质量属性。这一方法的基本特点是把任何形式的质量属性都具体化为场景，但**可修改性**是SAAM分析的主要质量属性。

(4) 风险承担者。SAAM 协调**不同参与者之间感兴趣的共同方面**，作为后续决策的基础，达成对架构的共识。

(5) 架构描述。SAAM 用于**架构的最后版本**，但**早于详细设计**。架构的描述形式应当被所有参与者理解。**功能、结构和分配**被定义为描述架构的3个主要方面。

(6) 方法活动。SAAM的**主要输入是问题描述、需求声明和架构描述**。下图描绘了SAAM分析活动的相关输入及评估过程。包括5个步骤，即**场景开发、架构描述、单个场景评估、场景交互和总体评估**。



淘宝搜索：文老师软考教育
浏览器输入网址：wenlaoshi.taobao.com

软件架构评估

(7) **已有知识库的可重用性**：SAAM 不考虑这个问题。

(8) 方法验证：SAAM 是一种成熟的方法，已被应用到众多系统中，这些系统包括空中交通管制、嵌入式音频系统、WRCS（修正控制系统、KWIC（根据上下文查找关键词系统）等。

◆架构权衡分析法ATAM

(1) 特定目标。ATAM 的目标是在**考虑多个相互影响的质量属性的情况下**，从原则上提供一种理解软件架构的能力的方法。对于特定的软件架构，在系统开发之前，可以使用ATAM方法确定在**多个质量属性之间折中的必要性**。

(2) 质量属性。开始时考虑的是系统的**可修改性、安全性、性能和可用性**。

(3) 风险承担者。在**场景、需求收集**相关活动中，ATAM方法需要所有系统相关人员的参与。

(4) 架构描述。架构描述基于**5 种基本结构来进行**，这5 种结构是从4+1 视图派生而来的。其中**逻辑视图**被分为**功能结构和代码结构**。这些结构加上它们之间适当的映射可以完整地描述一个架构。用一组**消息顺序图**表示运行时的交互和场景，对架构描述加以注解。

(5) 评估技术。使用了**场景技术**。从不同的架构角度，有3 种不同类型的场景，分别是**用例**（包括对系统典型的使用、引出信息）、**增长场景**（用于涵盖那些对它的系统的修改）、**探测场景**（用于涵盖那些可能会对系统造成过载的极端修改）。ATAM 还使用定性的**启发式分析方法**。

(6) 方法的活动。ATAM被分为4个主要的活动领域（或阶段），分别是**场景和需求收集、架构视图和场景实现、属性模型构造和分析、折中**。

软件架构评估

(7) 领域知识库的可重用性。领域知识库通过**基于属性的架构风格（ABAS）维护**。ABAS 有助于从架构风格的概念转向**基于特定质量属性模型**的推理能力。

(8) 方法验证。该方法已经应用到多个软件系统，但仍处在研究之中。虽然软件架构分析与评价已经取得了很大的进步，但是在某些方面也存在一些问题。例如，**架构的描述、质量特征的分析、场景不确定性的处理**、度量的应用架构分析和评价支持工具等，这些都影响和制约着分析评估技术的发展。

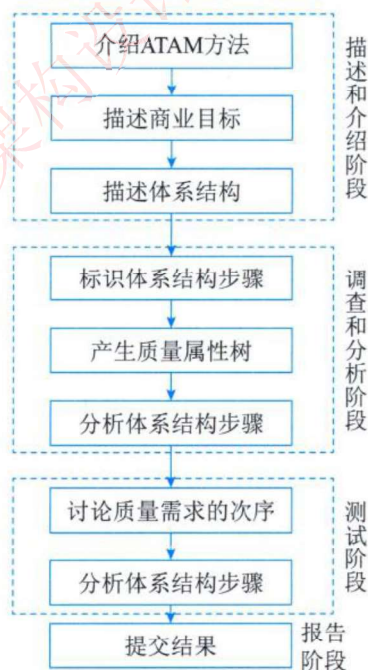
◆ ATAM方法采用效用树（Utility tree）这一工具**来对质量属性进行分类和优先级排序**。效用树的结构包括：**树根-质量属性-属性分类-质量属性场景（叶子节点）**。

◆ ATAM方法架构评估实践

◆ 注意：建议**写这个主题论文的，重点看一下第二版教材第8.3节，第289页**，书上给出了胡佛事件架构、银行事件架构这两个具体架构的详细描述，可作为大家写论文时的素材参考，不过书上写的比较多比较复杂，大家也要有选择性的参考。

◆ 用ATAM方法评估软件体系结构，其工作分为4个基本阶段，即**演示、调查和分析、测试和报告ATAM**。

淘宝搜索：文老师软考教育
浏览器输入网址：wenlaoshi.taobao.com



软件架构评估

◆ 成本效益分析法：用来**对架构设计决策的成本和收益进行建模**，是优化此类决策的一种手段。CBAM的思想就是架构策略影响系统的质量属性，反过来这些质量属性又会为系统的项目干系人带来一些收益，CBAM 协助项目干系人**根据其投资回报选择架构策略**。CBAM **在ATAM结束时开始，它实际上使用了ATAM 评估的结果**。分为以下8个步骤：

◆ 整理场景（确定场景，并确定优先级，选择三分之一优先级最高的场景进行分析）；

◆ 对场景进行细化（对每个场景详细分析，确定最好、最坏的情况）；

◆ 确定场景的优先级（项目干系人对场景投票，根据投票结果确定优先级）；

◆ 分配效用（对场景响应级别确定效用表，建立策略、场景、响应级别的表格）；

◆ 形成“策略-场景-响应级别的对应关系”；

◆ 确定期望的质量属性响应级别的效用（根据效用表确定所对应的具体场景的效用表）；

◆ 计算各架构策略的总收益；

◆ 根据受成本限制影响的投资报酬率选择架构策略（估算成本，用上一步的收益减去成本，得出收益，并选择收益最高的架构策略）。

软件架构评估

◆其他评估方法（仅了解）

1.SAEM方法。将软件架构看作一个最终产品以及设计过程中的一个中间产品，从外部质量属性和内部质量属性两个角度来阐述它的评估模型，旨在为软件架构的质量评估创建一个基础框架。

◆外部属性指用户定义的质量属性，而内部属性指开发者决定的质量属性。该软件架构评估模型包含以下几个流程。

（1）对待评估的质量属性进行规约建模。

（2）为外部和内部的质量属性创建度量准则，先从评估目的（如软件架构比较、最终产品的质量预测），评估角度（如开发者、用户、维护者），评估环境（架构作为最终产品或设计中间产品）出发来定义架构评估的目标，再根据目标相关的属性来提出问题，然后回答每个问题并提出相应的度量准则。

（3）评估质量属性，包括数据收集、度量和结果分析3个活动。

2.SAABNet方法。是一种用来表达和使用定性知识以辅助架构的定性评估。该方法来源于人工智能，允许不确定、不完整知识的推理。该方法使用BBN来表示和使用开发过程中的知识，包含定性和定量的描述，其中定性的描述是所有结点的图，定量的描述是每个结点状态相关的条件概率。其中的变量可分为3类，即架构质量属性变量（如可维护性、灵活性等）、质量属性的度量准则变量（如容错性、响应性等）和架构特征变量（如继承深度、编程语言等），高层抽象的质量属性变量分解为低层抽象的度量准则变量，度量准则变量则分解为更低层抽象的架构特征变量。

软件架构评估

3.SACMM方法。是一种软件架构修改的度量方法。

4.SASAM方法。通过对预期架构（架构设计阶段的相关描述材料）和实际架构（源代码中执行的架构）进行映射和比较来静态地评估软件架构。

5.ALRR方法。是一种软件架构可靠性风险评估方法，该方法使用动态复杂度准则和动态耦合度准则来定义组件和连接件的复杂性因素，其中，动态复杂度准则在某个场景的执行中分析组件的动态行为来度量组件的复杂性，动态耦合度准则在某个场景的执行中分析连接件的消息传递协议来度量连接件的复杂性。利用失效模式和影响分析（FMEA）技术。

6.AHP（层次分析法）方法。是对定性问题进行定量分析的一种简便、灵活而又实用的多准则决策方法。AHP方法的特点是把复杂问题中的各种因素通过划分为相联系的有序层次使之条理化，并在一般情况下通过两两对比，根据一定客观现实的主观判断结构，把专家意见和分析者的客观判断结果直接、有效地结合起来，将一定层次上元素的某些重要性进行定量描述，之后利用数学方法计算反映每一层次元素的相对重要性次序的权值，并最后通过所有层次之间的总排序计算所有元素的相对权重及对权重进行排序。

7.COSMIC+UML方法。基于度量模型来评估软件架构可维护性的方法。针对不同表达方式的软件架构，采用统一的软件度量COSMIC方法来进行度量和评估。该方法主要是为了辅助分析软件架构的演化方案是否可行，并在开源软件DCMMS的软件架构UML组件图上得以验证。

考试真题

体系结构权衡分析方法（Architecture Tradeoff Analysis Method，ATAM）包含 4 个主要的活动领域，分别是 场景和需求收集、体系结构视图和场景实现、（）、折中。基于场景的架构分析方法

（Scenarios-based Architecture Analysis Method，SAAM）的主要输入是问题描述、需求声明和（）。

A. 架构设计 B. 问题分析与建模 C. 属性模型构造和分析 D. 质量建模

A. 问题说明 B. 问题建模 C. 体系结构描述 D. 需求建模

答案：C，C

以下关于质量属性效用树的说法中错误的是（）

A. ATAM方法采用效用树这一工具来对质量属性进行分类和优先级排序

B. 效用树通过设计场景来评估质量属性

C. 效用树是一种重要的架构设计工具

D. 有3种不同的场景，分别是用例场景、增长场景、探测场景

答案：C

考试真题

构件相关：

以下关于构件的说法中，正确的是（）

A. 构件通常以二进制形式发布，无需编译；如果没有公开接口，则需要适配器

B. 构件模型提供了一组被构件使用的通用服务，这种服务包括两种：平台服务和通信服务

C. 构件在独立部署的时候是可以拆分的

D. 构件是一个实例单元，具有唯一的标志

答案：A

从构件的外部形态来看，构件可以分为五类，其中，（）具有高度的适应性和灵活性，能够根据不同的需求和环境进行调整和适应。（）是通过多个独立模块或子系统的组合构成的。每个模块可能独立开发，但最终通过装配形成一个完整的系统。

A. 独立而成熟的构件 B. 适应性构件 C. 装配的构件 D. 可修改的构件

A. 独立而成熟的构件 B. 适应性构件 C. 装配的构件 D. 可修改的构件

答案：B C

解析：ABCD+有限制的构件

淘宝搜索：文老师软考教育
浏览器输入网址：wenlaoshi.taobao.com

谢谢！

文老师软考教育

淘宝搜索：文老师软考教育
浏览器输入网址：wenlaoshi.taobao.com